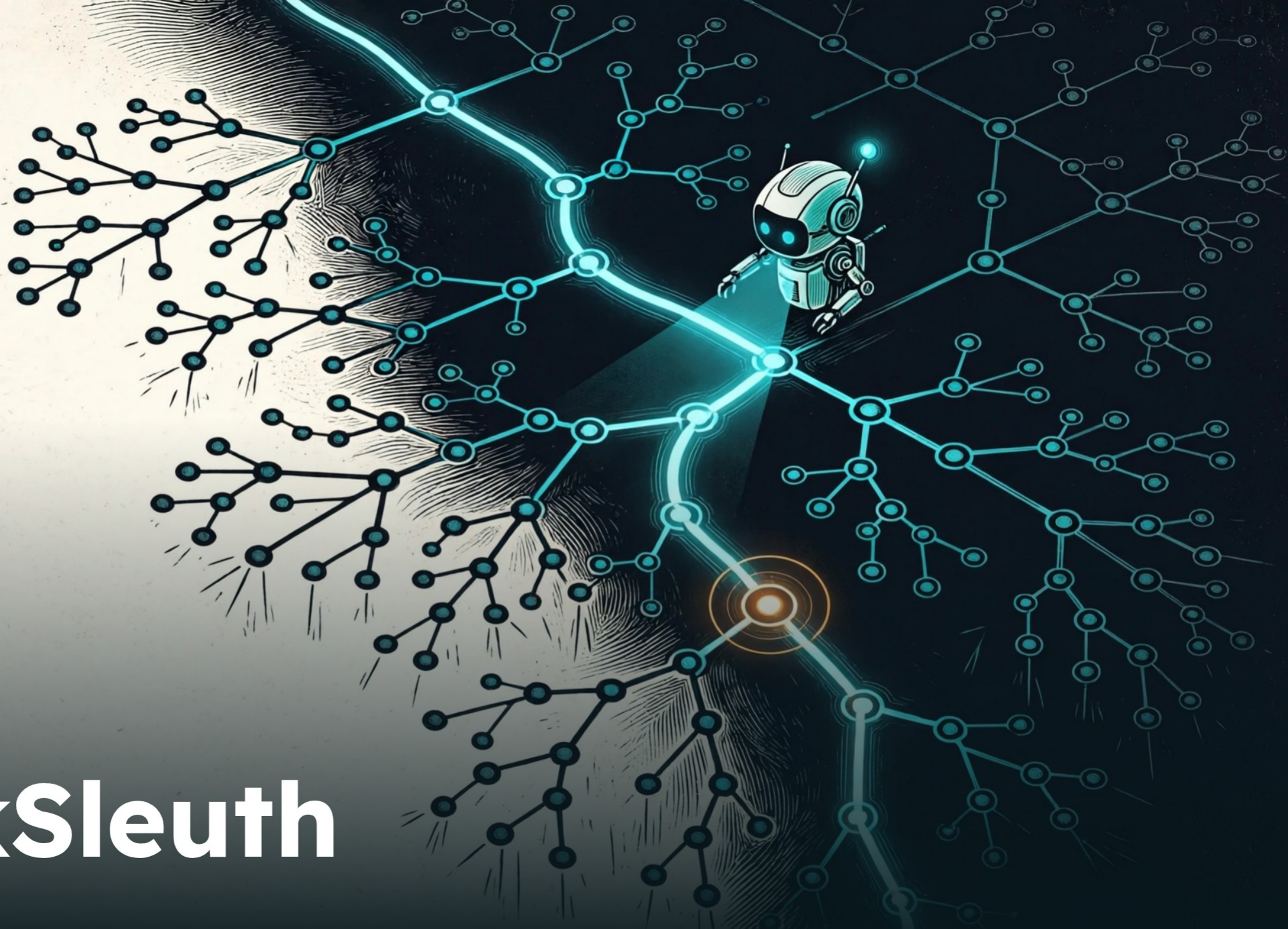




StackSleuth

Regression debugging taxes every release

30 min–3
hrs

Senior-engineer time spent chasing a single broken test.

1

Reproduce

Separate a real regression from a flaky test.

2

Search

Walk commit history and guess where behavior changed.

3

Inspect

Read diffs and reconstruct intent.

4

Patch

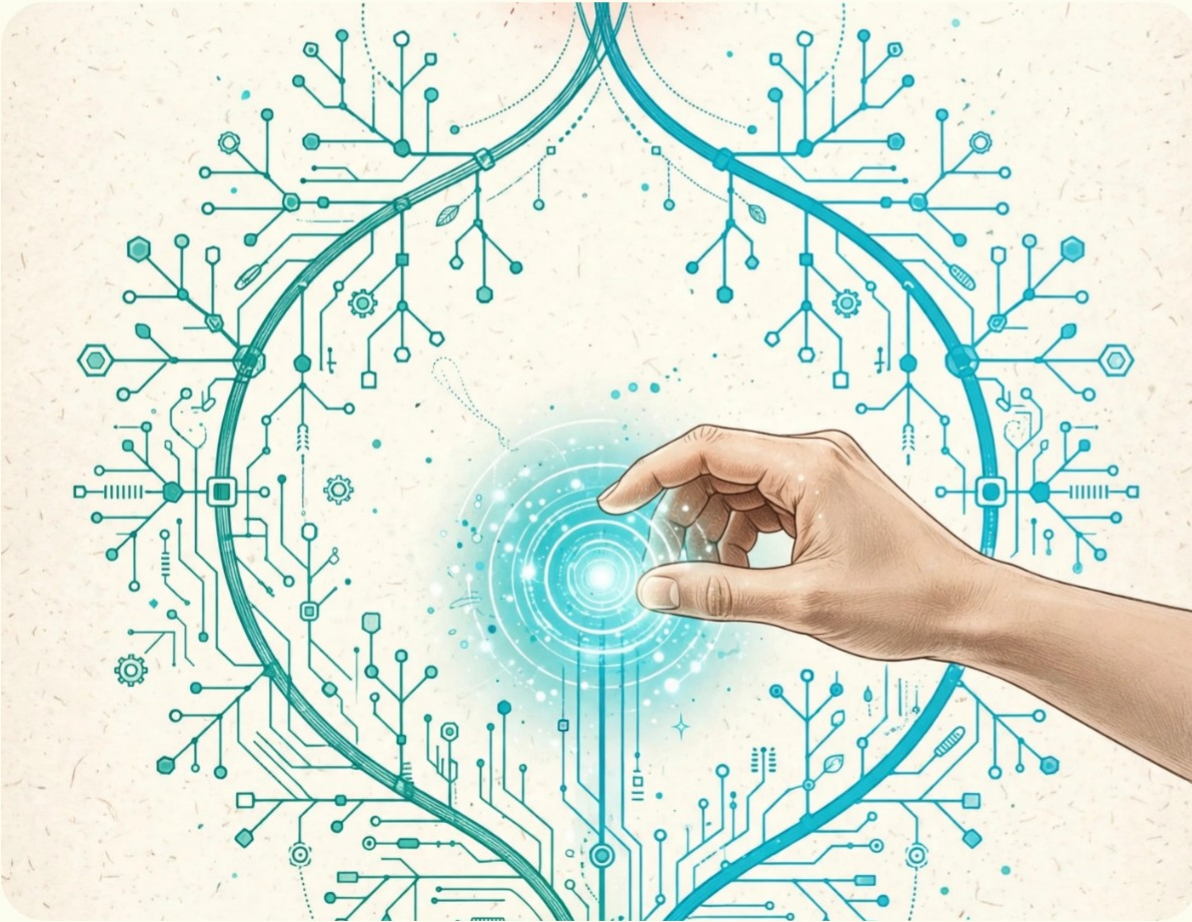
Craft the smallest safe correction.

5

Verify

Re-run the full suite and preserve the evidence.

Automation moves fast. The engineer keeps control.



IBM Bob reasons

Bob calls StackSleuth's MCP tools and interprets each result.

Git bisect proves

The culprit commit comes from evidence, not a model guess.

Human approval gates

The proposal is held until an engineer reviews the diff and rationale.

Evidence persists

Every stage lands in an append-only debugging trail.

Six stages turn one failure into a verified fix

01	Detect	Run the suite and identify the failing test.	run_tests
02	Reproduce	Run the failure three times in isolation.	reproduce
03	Bisect	Search history until the first bad commit is proven.	bisect
04	Inspect	Expose the culprit diff and root cause.	inspect_commit
05	Propose	Save a minimal patch and rationale without applying it.	propose_patch
06	Apply and verify	After approval, apply the patch and prove the suite is green.	apply_patch

The demo proves the loop twice

2 steps

Bulk discount boundary

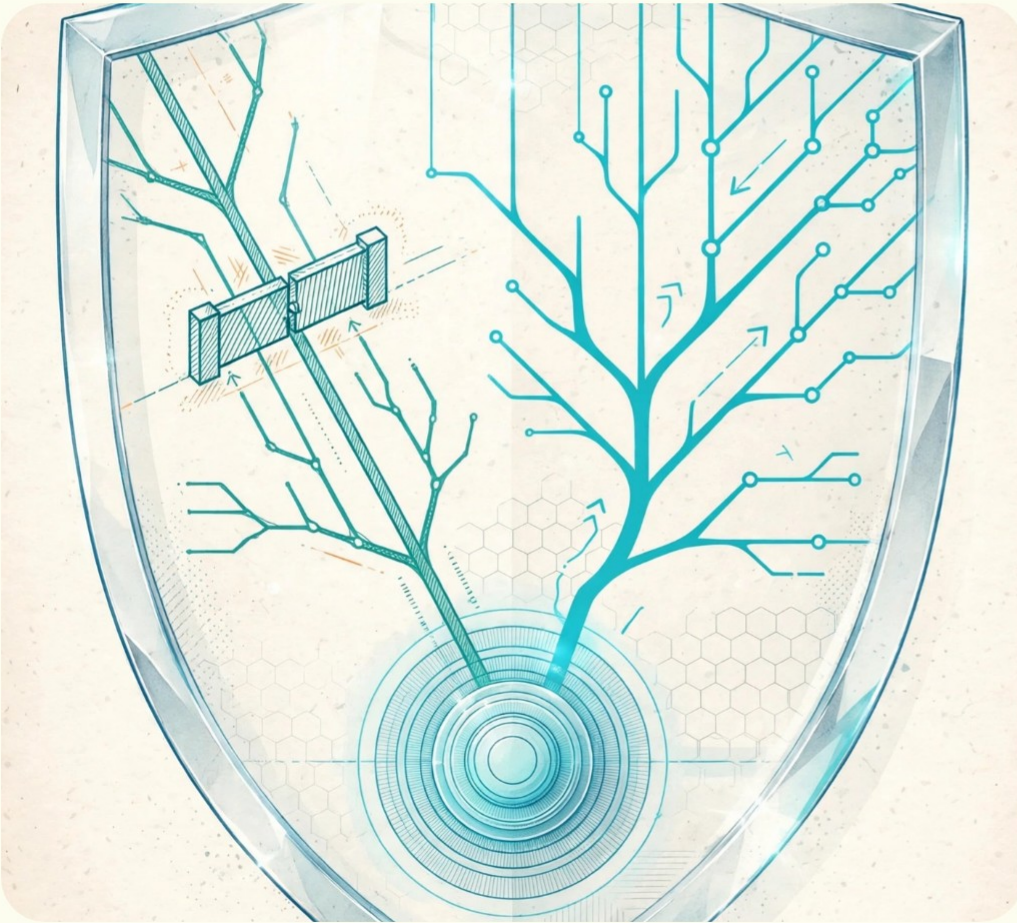
- Culprit: `afcc8ee`
- Root cause: `qty > 10` excluded exactly 10 units
- Minimal fix restored the inclusive boundary
- **10/10 tests green**

SAVE10

Second bug variant

- Two failing coupon tests reproduced 3/3
- Culprit found in one bisect step
- Coupon rate restored from 5% to 10%
- **10/10 tests green**

Guardrails stop false greens and unsafe bisects



Collection errors stay red

Pytest crashes and import failures are identified explicitly instead of masquerading as zero failed tests.

Dirty trees stop before bisection

Uncommitted changes trigger a refusal with clear stash-or-commit guidance.

9 hardening tests passed

StackSleuth converts debugging toil into an auditable workflow

Speed

Automates reproduction, history search, inspection, and verification.

Correctness

Bisect establishes the culprit before any fix is proposed.

Safety

A human reviews the patch before it can change code.

Auditability

An append-only evidence log preserves the complete decision chain.

Built for teams where failure costs time

Engineering teams reclaim feature time.

SRE and on-call get an automated first responder.

Regulated teams retain a reviewable code-change trail.



**Find the regression.
Prove the cause.
Ship the safe fix.**